

test_rtnorm

Multivariate Normal Tests

Mardia's test

This test is based on skewness and kurtosis. Given we observe k samples of $X_1, X_2, \dots, X_k$ we compute

$$\begin{aligned}\hat{\Sigma} &= \frac{1}{n} \sum_{j=1}^n (x_j - \bar{x})(x_j - \bar{x})^T \\ A &= \frac{1}{6n} \sum_{i=1}^n \sum_{j=1}^n \left[(x_i - \bar{x})^T \hat{\Sigma}^{-1} (x_j - \bar{x})^T \right]^3 \\ B &= \sqrt{\frac{n}{8k(k+2)}} \left\{ \frac{1}{n} \sum_{i=1}^n \left[(x_i - \bar{x})^T \hat{\Sigma}^{-1} (x_i - \bar{x}) \right]^2 \right\}\end{aligned}$$

Under $H_0 : X \sim N_p(\mu, \sigma)$, $A \sim \chi_{1/6k(k+1)(k+2)}^2$ and $B \sim N(0, 1)$.

```
MVN::mardia(iris[1:50, 1:4])
```

```
##           Test Statistic   p.value    Method
## 1 Mardia Skewness 25.664345 0.1771859 asymptotic
## 2 Mardia Kurtosis  1.294992 0.1953229 asymptotic
```

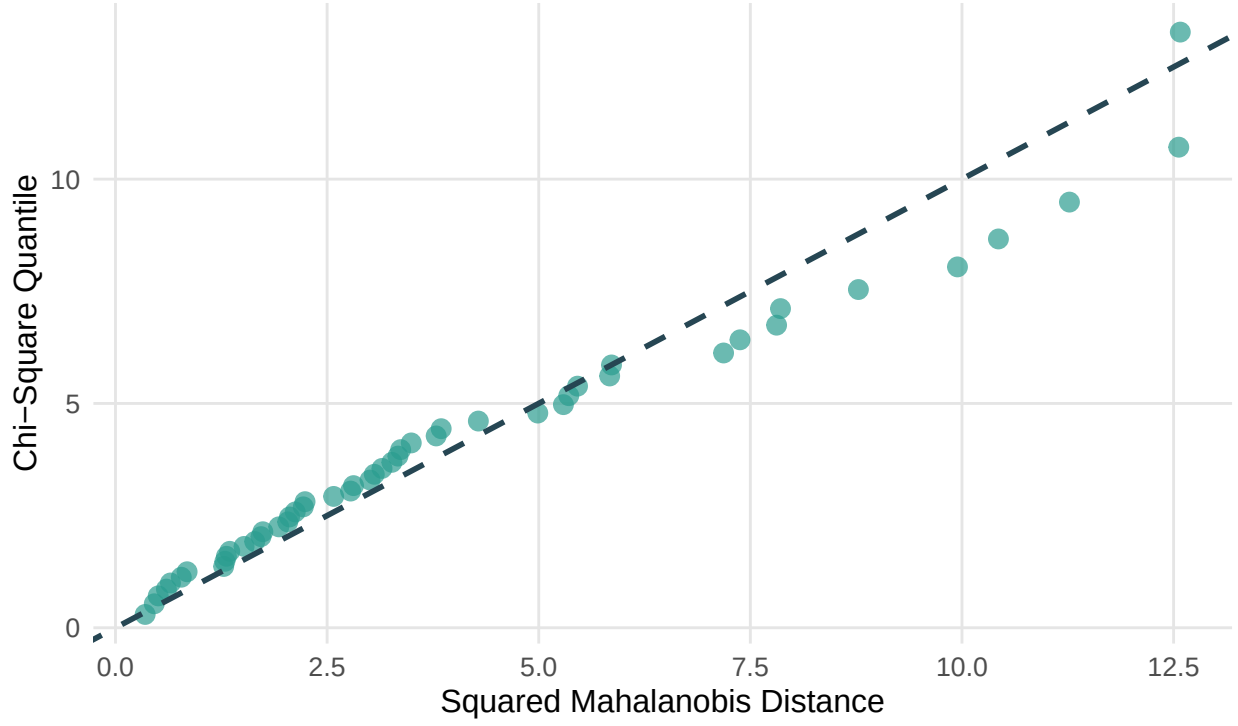
Malahanobis Distance QQ Plot

The variable $d^2 = (x - \mu)^T \Sigma^{-1} (x - \mu) \sim \chi_p^2$. We can plot the ordered Malahanobis distances versus the esimated quantiles on a QQ plot.

```
MVN::multivariate_diagnostic_plot(iris[1:50, 1:4])
```

Mahalanobis Q-Q Plot

Squared distances vs. chi-square quantiles



Eigenvalue

Assume we observe n samples of $X_1, X_2, \dots, X_n$ and that the eigenvalues of Σ are distinct and positive such that $\lambda_1 > \lambda_2 > \dots > \lambda_p > 0$. Denote the vector of eigenvalues as λ and the corresponding eigenvectors as $\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_p$.

Let Λ is a diagonal matrix of the eigenvalues. Then $\sqrt{n}(\hat{\lambda} - \lambda) \simeq N_p(0, 2\Lambda^2)$

Let

$$E_i = \lambda_i \sum_{k=1, k \neq i}^p \frac{\lambda_k}{(\lambda_k - \lambda_i)^2} \mathbf{e}_k \mathbf{e}_k^T$$

. Then $\sqrt{n}(\hat{\mathbf{e}}_i - \mathbf{e}_i) \simeq N_p(0, E_i)$.

Each $\hat{\lambda}_i$ is distributed independently of elements of associated $\hat{\mathbf{e}}_i$.

However, in practice, this large sample approximation converges very slowly. The asymptotic distributions depend on the true eigenstructure of Σ too.

Matrix Variate Normal Tests

Mahalanobis Distance QQ Plot

We can calculate the Mahalanobis Distance for a multivariate normal model as

$$D(\mathbf{X}_i, \hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}}) = (\text{vec}(\mathbf{X}_i) - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\text{vec}(\mathbf{X}_i) - \boldsymbol{\mu})$$

.

If a Kronecker product structure exists for Σ , then

$$D(\mathbf{X}_i, \hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}}) = D_M(\mathbf{X}_i, \mathbf{M}, \mathbf{V}, \mathbf{U})$$

.

I was wondering how this would be generalizable for the tensor variate normal. For example, if each X_i draw is a $2 \times 3 \times 4$. Then we would have three covariance estimates: $\boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2, \boldsymbol{\Sigma}_3$. Instead of using the matrix product, we would use the n -mode product between the draws and the covariance matrices.

Since we take the trace of the matrix product, we only care about the diagonals of the resulting object. I am unsure what the generalization of that in the tensor case would be. The structure reminds me of the hyperdiagonal tensor used in the PARAFAC decomposition though.

Would we calculate the Mahalanobis Distance for a tensor variate normal model as something like the following?

$$D_T(\mathcal{X}_i, \mathcal{M}, \mathbf{V}, \mathbf{U}) = \text{tr}\{(\mathcal{X}_i - \mathcal{M}) \times_1 \boldsymbol{\Sigma}_1^{-1} \times_2 \boldsymbol{\Sigma}_2^{-2} \times_3 \boldsymbol{\Sigma}_3^{-1} (\mathcal{X}_i - \mathcal{M})^T\}.$$

```
n <- 1e3

s1 <- matrix(rnorm(4), nrow = 2); s2 <- matrix(rnorm(9), nrow = 3);
s1 <- crossprod(s1, s1);
s2 <- crossprod(s2, s2);
list_sigmas <- list(s1, s2)
rand_mu <- array(rnorm(6, mean = 10), dim = c(2, 3))
rtnorm_draws <- rtnorm(n = n, mu = rand_mu, sigmas = list(s1, s2))

draws <- matrix(0, nrow = n, ncol = 6)

for(i in 1:n) {
  draws[i, ] <- c(rtnorm_draws[i, , ])
}

mu_vec <- apply(draws, MARGIN = 2, FUN = mean)

vec_dist <- mahalanobis(draws, center = mu_vec, cov = cov(draws))

est_norm <- mle_est(rtnorm_draws)

mean_mat <- est_norm$mu

U <- est_norm$sigmas[[1]]
V <- est_norm$sigmas[[2]]

mat_dist <- rep(0, 1000)

c_hat <- sum(diag(cov(draws)))/sum(diag(kronecker(U, V)))

for(i in 1:1000) {
  Xi <- rtnorm_draws[i, , ] - mean_mat
  mat_dist[i] <- sum(diag(solve(U) %*% Xi %*% solve(V) %*% t(Xi)))
}

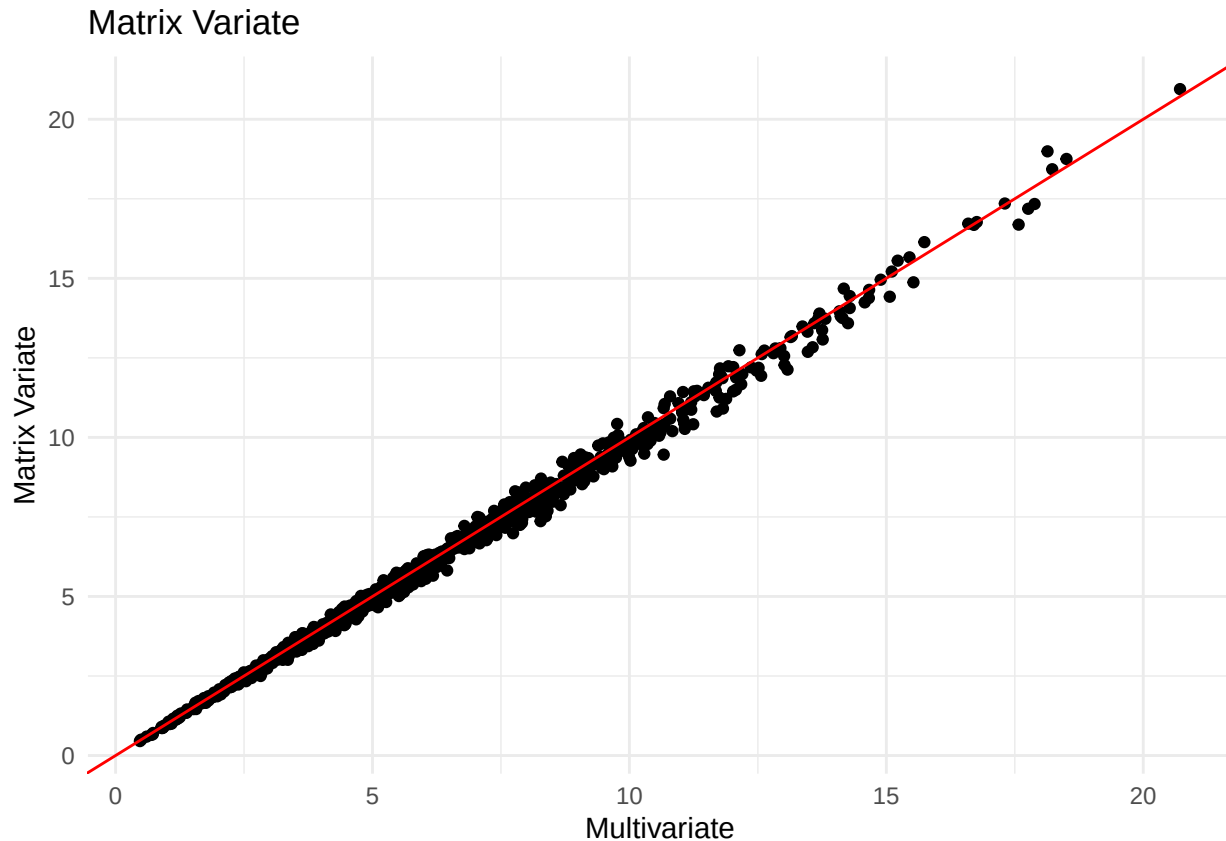
mat_dist <- mat_dist / c_hat

tibble("vec" = (vec_dist),
```

```

    "mat" = (mat_dist)) |>
  ggplot() +
  geom_point(aes(x = vec, y = mat)) +
  geom_abline(slope = 1, color = "red") +
  labs(x = "Multivariate", y = "Matrix Variate",
       title = "Matrix Variate") +
  theme_minimal()

```



```

Sigma <- matrix(c(
  2, 1, 0.5, 0.1, 0, 0.5,
  1, 3, 1, 0.3, 0.2, 0.4,
  0.5, 1, 4, 0.5, 0.1, 0.7,
  0.1, 0.3, 0.5, 2, 0.8, 0.2,
  0, 0.2, 0.1, 0.8, 3, 1.2,
  0.5, 0.4, 0.7, 0.2, 1.2, 5
), nrow = 6, byrow = TRUE)

# mean for vec form
mu_vec <- rep(0, 6)

# generate vectorized matrices
n <- 1000
X_vec <- mvrnorm(n, mu = mu_vec, Sigma = Sigma)

# reshape into 2x3 matrices
X_array <- array(0, dim = c(n, 2, 3))
for(i in 1:n) {

```

```

  X_array[i,,] <- matrix(X_vec[i,], nrow = 2)
}

rtnorm_draws <- X_array

draws <- matrix(0, nrow = n, ncol = 6)

for(i in 1:n) {
  draws[i, ] <- c(rtnorm_draws[i, , ])
}

mu_vec <- apply(draws, MARGIN = 2, FUN = mean)

vec_dist <- mahalanobis(draws, center = mu_vec, cov = cov(draws))

est_norm <- mle_est(rtnorm_draws)

mean_mat <- est_norm$mu

U <- est_norm$sigmas[[1]]
V <- est_norm$sigmas[[2]]

mat_dist <- rep(0, 1000)

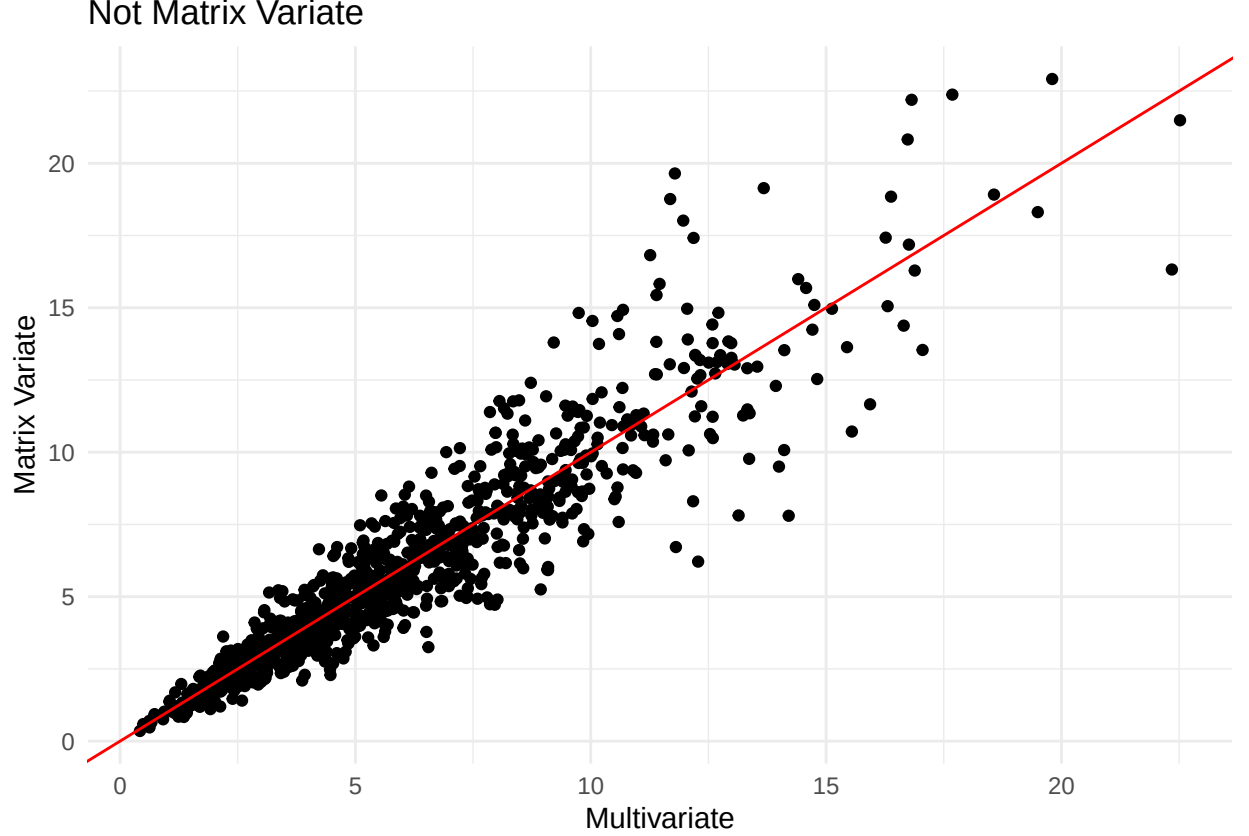
c_hat <- sum(diag(cov(draws)))/sum(diag(kronecker(U, V)))

for(i in 1:1000) {
  Xi <- rtnorm_draws[i, , ] - mean_mat
  mat_dist[i] <- sum(diag(solve(U) %*% Xi %*% solve(V) %*% t(Xi)))
}

mat_dist <- mat_dist / c_hat

tibble("vec" = (vec_dist),
       "mat" = (mat_dist)) |>
  ggplot() +
  geom_point(aes(x = vec, y = mat)) +
  geom_abline(slope = 1, color = "red") +
  labs(x = "Multivariate", y = "Matrix Variate", title = "Not Matrix Variate") +
  theme_minimal()

```



Likelihood Ratio Tests for Sigma

Using the paper Likelihood Ratio Tests for High-Dimensional Normal Distributions by Jiang and Qi (2015), they suggest a spherical test. For a multivariate normal distribution $N_p(\mu, \Sigma)$ where $\mu \in \mathbb{R}^p$ is a mean vector and Σ is a $p \times p$ covariance matrix, we consider this spherical test.

$$H_0 : \Sigma = \lambda \mathbf{I}_p \text{ vs } H_a : \Sigma \neq \lambda \mathbf{I}_p$$

with unspecified λ .

Suppose $\mathbf{x}_1, \dots, \mathbf{x}_n$ are iid $\mathbb{R}^p$ from the normal distribution. As usual, set

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \text{ and } \mathbf{S} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T.$$

Then the LRT statistic for this test is

$$V_n = |\mathbf{S}| \left(\frac{\text{tr}(\mathbf{S})}{p} \right)^{-p}.$$

Then $-(n-1)\rho \log V_n$ converges to χ_f^2 as $n \rightarrow \infty$ where

$$\rho = 1 - \frac{2p^2 + p + 2}{6(n-1) * p} \text{ and } f = \frac{1}{2}(p-1)(p+2).$$

```

test_sigmas <- function(est_sig) {
  p_vals <- rep(0, length(est_sig))

  for(d in 1:length(est_sig)) {
    curr_sig <- est_sig[[d]]
    p <- dim(curr_sig)[[1]]

    rho <- 1 - (2 * p^2 + p + 2)/(6 * (n-1) * p)
    f <- 1/2 * (p - 1) * (p + 2)

    Vn <- det(curr_sig) * (sum(diag(curr_sig))/p)^(-p)

    test_stat <- -(n - 1) * rho * log(Vn)

    p_vals[d] <- pchisq(q = test_stat, df = f, lower.tail = FALSE)
  }

  p_vals
}

n <- 1e3

```

Test with sigma 1 being diagonal. Since the null is that the sigma is diagonal, we expect to fail to reject for the first sigma.

```

s1 <- diag(2); s2 <- matrix(rnorm(9), nrow = 3); s3 <- matrix(rnorm(16), nrow = 4)
s1 <- 4 * s1
s2 <- crossprod(s2, s2); s3 <- crossprod(s3, s3)
list_sigmas <- list(s1, s2, s3)
rand_mu <- array(rnorm(24, mean = 10), dim = c(2, 3, 4))
rtnorm_draws <- rtnorm(n = n, mu = rand_mu, sigmas = list(s1, s2, s3))

est_mle <- mle_est(rtnorm_draws)

test_sigmas(est_mle$sigmas)

```

```
## [1] 0.985153 0.000000 0.000000
```

Test with sigma 1, 2, 3 being diagonal

```

s1 <- diag(2); s2 <- diag(3); s3 <- diag(4)
s1 <- 4 * s1
s2 <- 2 * s2
s3 <- 8 * s3
rand_mu <- array(rnorm(24, mean = 10), dim = c(2, 3, 4))
rtnorm_draws <- rtnorm(n = n, mu = rand_mu,
  sigmas = list(s1, s2, s3))

est_mle <- mle_est(rtnorm_draws)
test_sigmas(est_mle$sigmas)

```

```
## [1] 0.9704249 0.9946918 0.9994268
```

Test with sigma 2 being diagonal with some error

```

s1 <- diag(2); s2 <- diag(3); s3 <- diag(4)
s1 <- 4 * s1

```

```

s2 <- 2 * s2 + rnorm(9, mean = 0.4, sd = 0.2)
s3 <- 8 * s3
rand_mu <- array(rnorm(24, mean = 10), dim = c(2, 3, 4))
rtnorm_draws <- rtnorm(n = n, mu = rand_mu,
                      sigmas = list(s1, s2, s3))

est_mle <- mle_est(rtnorm_draws)
test_sigmas(est_mle$sigmas)

```

```
## [1] 8.865649e-01 2.559849e-28 9.988050e-01
```

Test with sigma 1 and sigma 3 being diagonal

```

s1 <- diag(2); s2 <- diag(3); s3 <- 3 * diag(4)
s1 <- 4 * s1
s2 <- 2 * s2 + rnorm(9, mean = 0.4, sd = 0.2)
s3 <- 8 * s3
rand_mu <- array(rnorm(24, mean = 10), dim = c(2, 3, 4))
rtnorm_draws <- rtnorm(n = n, mu = rand_mu,
                      sigmas = list(s1, s2, s3))

est_mle <- mle_est(rtnorm_draws)
test_sigmas(est_mle$sigmas)

```

```
## [1] 9.278984e-01 2.803435e-09 9.918052e-01
```

Test the power when sigma 1 and sigma 3 are diagonal

```

rt_power <- tibble("p1" = rep(0, 1000), "p2" = rep(0, 1000), "p3" = rep(0, 1000))

list_mles <- vector("list", 3)

for(i in 1:100) {
  rtnorm_draws <- rtnorm(n = n, mu = rand_mu,
                        sigmas = list(s1, s2, s3))

  est_mle <- mle_est(rtnorm_draws)

  list_mles[[i]] <- est_mle$sigmas

  curr_test <- test_sigmas(est_mle$sigmas)

  rt_power[i, "p1"] <- curr_test[1]
  rt_power[i, "p2"] <- curr_test[2]
  rt_power[i, "p3"] <- curr_test[3]
}

rt_power$p2[1:100] |>
  as_tibble() |>
  ggplot() +
  geom_histogram(aes(x = value))

```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

